

Лабораторна робота №3

CI/CD

Мета роботи

Метою завдання є практичне закріплення знань з таких тем курсу:

- Continuous Integration
- Continuous Delivery

Студент повинен налаштувати автоматичне тестування, розгортання та валідацію проекту з [Лабораторної роботи №1](#).

Завдання

Опис завдання

Необхідно налаштувати автоматичне тестування та розгортання проекту з першої лабораторної роботи. Проект повинен проходити статичний аналіз коду, автоматичні тести та збірку у контейнер. При успіху — автоматично розгортати нову версію проекту на віртуальній машині за допомогою [self-hosted runner](#). Після розгортання — верифікувати, що проект працює.

Аналіз коду

Виконати аналіз коду за допомогою лінтерів. Потрібно перевірити на наявність проблем:

- Код застосунку із [Лабораторної роботи №1](#)
- Скрипти автоматизованого розгортання

Наприклад: для аналізу проекту ми можемо використовувати лінтери, специфічні для мови програмування, такі як ESLint, Flake8, MyPy, тощо. Для аналізу Dockerfile — Hadolint. Для аналізу shell-скриптів — Shellcheck, для аналізу конфігураційних файлів — YamlLint, JSON Schema, тощо.

Налаштувати автоматичний запуск аналізу на кожен коміт в основну гілку репозиторія, на ановані теги, а також на кожен Pull Request в основну гілку репозиторія.

Автоматичні тести

Додати до веб-застосунку набір тестів, які перевірятимуть коректність його роботи. Налаштувати перевірку покриття коду тестами. Вважати тестування неуспішним, якщо покриття коду тестами менше 40%.

Налаштувати автоматичний запуск цього набору тестів на кожен коміт в основну гілку репозиторія, на анотовані теги, а також на кожен Pull Request в основну гілку репозиторія.

[Налаштувати правила для Pull Request](#), які забороняють злиття, якщо аналіз коду та тести не пройшли успішно.

Завантажувати актуальний звіт по покриттю коду тестами в основній гілці репозиторію у якості артефакту.

Збірка образу

Налаштувати автоматичну збірку production-образу з вашим застосунком на кожен коміт в основну гілку репозиторія та на анотовані теги за умови, що аналіз коду та автоматичні тести пройшли успішно. Зібрані образи публікувати в [GitHub Container Registry](#) вашого репозиторію.

Образи тегувати наступним чином:

- на коміт в гілку: latest, sha-<full-commit-hash>
- на анотовані теги: stable, <tag>

Налаштування окремого раннеру для розгортання

Оскільки проєкт розгортається локально на віртуальній машині, вам необхідно, щоб GitHub-Runner, на якому ви реалізовуватимете розгортання проєкту мав доступ до локальних ресурсів. Для цього вам потрібно підготувати ще одну віртуальну машину (за основу візьміть образ [Ubuntu 24.04 Server](#)) та налаштувати на ній [self-hosted runner](#) для вашого репозиторію на GitHub.

Важливо: враховуючи те, що для задачі лабораторних робіт ми використовуємо публічні репозиторії, а документація GitHub рекомендує використання self-hosted ранерів тільки для приватних репозиторіїв (з точки зору безпеки), то робимо наступним чином:

- Піднімаємо vm
- Налаштовуємо раннер. В ідеалі — готуємо для цього автоматизований скрипт, який поставить усе необхідне. Якщо скрипт зробити не вдається — пишемо інструкцію, як це зробити. При цьому важливо **не додавати** приватні токени для реєстрації ранера в репозиторій — тобто етап підключення ранера до репозиторію робимо **виключно вручну**.
- Проганяємо усі необхідні конвеєри (pipelines), необхідні для виконання та демонстрації виконання лабораторної роботи

- Після завершення — зупиняємо віртуальну машину з раннером (або навіть видаляємо її), щоб інші не могли нею скористатися і скомпрометувати ваші системи

Також важливо налаштувати доступ (наприклад, ssh) з вашого раннера на віртуалку, на яку ви будете розгортати проект. **Розгортати проект на віртуальну машину-раннер категорично заборонено.**

Розгортання проекту

Створити скрипт, який на віртуалці з [Ubuntu 24.04](#) налаштує середовище для вашого застосунку (база даних, nginx, docker daemon). За основу цього скрипта можна взяти скрипт автоматичного розгортання із [Лабораторної роботи №1](#). За допомогою цього скрипта налаштувати віртуальну машину (в подальшому target node).

Налаштувати автоматичне розгортання нової версії вашого проекту на кожен анотований тег за умови успішного проходження аналізу коду, тестування та збірки образу. Розгортання повинне відбуватися з ранера, а застосунок розгортатися в контейнері на віртуальній машині target node.

У [Лабораторній роботі №1](#) був акцент на systemd та socket activation. В цій лабораторній роботі застосунок розгортається в контейнері. Для керування контейнером (start/stop/restart, тощо) необхідно написати systemd-unit. Для спрощення, в цій роботі socket activation можна не використовувати і не реалізовувати.

Верифікація

На self-hosted ранері після розгортання запустити процес верифікації, який перевірить:

- Доступність сервісу
- Коректність налаштування nginx (вимоги до налаштування nginx — в [Лабораторній роботі №1](#))
- Опціонально — будь-які інші перевірки коректності розгортання на ваш розсуд.

Для верифікації коректності розгортання дозволяється використовувати як скриптування за допомогою shell-скриптів, так і використання спеціалізованих інструментів, наприклад, [testinfra](#).

Опціонально у випадку неуспішної верифікації налаштувати сповіщення собі на email або в будь-який месенджер.

Демонстрація

В репозиторії має бути продемонстровано наступне:

- PR, який був успішно злитий в основну гілку після проходження усіх перевірок
- PR, який не може бути злитий в основну гілку, бо не пройшли автоматизовані тести/аналіз

- Лог успішного розгортання проекту і успішної верифікації розгортання
- Лог успішного розгортання проекту і неуспішної верифікації розгортання
- Звіт по покриттю коду тестами

Результат роботи

Git репозиторій

Код має бути розміщений на GitHub. Репозиторій не повинен містити чутливої інформації, такої як логіни/паролі до БД розгорнутого інстансу, тощо (за потреби використовуйте для такої інформації [GitHub Secrets](#)).

Налаштування віртуальної машини target node окрім паролей користувачів має бути відтворюване за допомогою скрипта в репозиторії і CD-задачі.

Історія комітів повинна бути логічною та зрозумілою, відображати хід виконання роботи студентом.

Міні-звіт

Оскільки GitHub має обмежений термін зберігання логів, про всяк випадок підготуйте міні-звіт зі скріншотами і текстовими логами із розділу [Демонстрація](#). Цей міні-звіт також прикріпіть у classroom.